This is a copy of a conversation between ChatGPT & Anonymous.

You said:

I am now doing Special Participation E.

ChatGPT said:

Great — what do you need for Special Participation E?

You said:

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

I just gave you my notes for the class, as well as all the previous questions I could fine, which mostly come from Homeworks and Discussions (a lot of which appeared on previous exams and are meant to be a strong reference for the content we can expect on this upcoming exam); I haven't yet gotten access to Spring 2025's final, but we are preparing for the Fall 2025 final. Some more information, there were 4 review sessions. The first was on optimizers, particularly SGD, muP, norms, initialization, and Muon being the focus. The next 3 are on: (a) Attention and Transformers, (b) Post-Training, (c) Diffusion Models. The exam will be entirely written, with no actual coding, and it will be entire paper based. It is possible that there will be "code" questions where we read code on paper and debug it with pen/pencil, but we obviously aren't running any actual code or writing very much. Most of the exam will almost certainly be focused on concepts and math. I want you to first lay out the concepts we are likely to face. Just because the 4 review sessions targeted optimizers, attention/transformers, post-training, and diffusion, does not mean stuff like initialization, CNNs, GNNs, RNNs, SSMs, etc are ruled out; there will almost definitely at least a bit of each on the final. Then, design some exam questions that target each topic approximately proportionally to the predicted coverage of each topic on the exam. There should be a gradient of difficulty, in that some questions are easier (conceptual, multiple choice, fill in a table, etc), some are medium (basic theory or debugging, conceptual math, diagrams), and some are harder (full

algebraic/computational questions). These questions will serve as a sample baseline for this chat, which will act as an exam prep tutor for the course.

ChatGPT said:

Perfect — here is **the full conceptual landscape** of what the Fall 2025 EECS 182 final is likely to test, based on:

- Your lecture notes (which you uploaded earlier)
- Prior exams (SP23 final, midterm, SP23 MT)
- Homeworks and discussions (especially HW12 and HW13, which align with the 4 review sessions)
- The four official review topics: **optimizers**, **attention/transformers**, **post-training**, **diffusion**

I'll first map out the *full conceptual territory* likely to appear, then give you **a well-balanced exam-style question set** spanning easy → medium → hard difficulty for each topic, weighted by likelihood.

I. FULL CONCEPTUAL MAP OF LIKELY EXAM CONTENT

What follows is a *clean, structured list* of everything that is reasonably in-scope for the written final.

---

1. Optimization & MuP (very likely)

Topics from lecture + review session + HW12/13:

Foundations

- SGD, momentum, Adam, RMSProp
- Update equations, bias correction, stability
- Exploding/vanishing gradients and mitigations
  (init scaling, He/Xavier, residual connections, normalization)

muP (Maximal Update Parametrization)

- Why standard parametrization breaks width scaling

- Separation of **parameter shapes** vs **update magnitudes**
- What muP fixes: learning-rate stability under width scaling
- Base vs. superposition networks
- Classification head scaling
- Consistency of gradients as width → ∞
- How to recognize *incorrect* MuP scaling in code

Muon optimizer (review session emphasis)

- Norm-preserving updates
- Per-neuron vs. per-weight norm control
- Why Muon eliminates many learning rate tuning issues

Likely exam styles

- Debugging an optimizer implementation
- Choosing learning rates under MuP vs SP
- Compute gradient flow or parameter norm drift
- Identify breaking points where MuP assumptions fail

---

2. Initialization & Norm Dynamics (likely)

- Xavier vs. He initialization
- Variance propagation through linear/ReLU layers
- Why bad init → exploding/vanishing activations
- What batchnorm does to distributions
- What LayerNorm does (and how it differs)
- Distribution transformation diagrams (like midterm Q3)

---

3. RNNs, SSMs, GNNs, CNNs (medium likelihood)

These appear on almost every 182 final in some form.

RNNs

- Backprop through time, weight sharing
- Gradient explosion/decay through repeated multiplication
- Computing ∂y/∂w in simple linear RNNs (like midterm Q5)
- Long-range dependencies vs. attention

SSMs (S4, Mamba)

- Continuous → discrete conversion idea
- Why SSMs have better long-context scaling
- Comparison to RNN recurrence vs attention
- Convolution-like interpretation

CNNs

- Shape calculations, padding, stride
- Parameter counting
- Translation equivariance
- Global average pooling invariances
- Problems like midterm CNN question (x1,x2,x3,x4)

GNNs

- Message passing constraints
- Permutation invariance requirements
- Valid vs invalid update functions
- Failure cases (over-smoothing, expressive limits)

---

4. Attention & Transformers (very likely)

Attention math

- $QK^T / \sqrt{d}$ scaling
- Softmax pooling interpretation

- Why softmax → differentiability
- Why argmax attention fails (midterm Q6)

Transformers

- Multi-head projection shapes
- Masking in decoder
- Residual connections
- LayerNorm placement (pre-LN vs post-LN)
- Weight sharing (like in HW12 bug detection)

Positional embeddings

- Absolute vs relative
- Learned vs sinusoidal

Common exam tasks

- Debugging a transformer forward pass
- Tracing tensor shapes
- Showing a masking pattern
- Reasoning about parameter initialization

---

5. Post-Training (SFT, RLHF, Reward Models, DPO) — very likely

From HW13 and review session.

SFT

- Cross-entropy training
- Distribution shift issues

Reward Modeling

- Bradley-Terry preference likelihood
- Why reward differences → logits
- Normalization of reward models

RLHF

- PPO objective
- KL penalty as regularizer
- Collapse risks

DPO (Direct Preference Optimization)

- Derivation from KL-regularized objective
- Cancellation of partition function Z(x)
- Why this is computationally easier
- Relationship to optimal policy form (HW13, Q2)

---

6. Diffusion (DDPM, DDIM) — very likely

From HW13.

Forward process

- Gaussian noise accumulation
- q(x_t | x_0) closed-form
- Variance scaling across time

Reverse process

- Mean and variance of p(x_{t-1} | x_t)
- Telescoping products (HW13 Q1d/e)
- Why noise must be added to recover correct variance
- DDPM vs DDIM deterministic step differences
- How to train a noise-predictor or x_0-predictor

Likely tasks

- Simplify Gaussian conditional variance
- Compute telescoping product in reverse diffusion
- Convert Riemann sum → integral (Δt→0)

- Explain why removing noise collapses variance (HW13)

---

7. Generative Models & KL Divergence (medium-likely)

- Forward vs reverse KL
- Mode-covering vs mode-seeking
- Examples where KL(P‖Q) finite but KL(Q‖P)=∞
- Matching priors (VAE KL term)
- Reparameterization trick

---

8. Meta-Learning (MAML) (medium-low likelihood)

- Inner loop vs outer loop
- Why rapid adaptation works
- Gradient through gradient update
- Simplified one-step regression interpretation (HW12 last section)

SUMMARY OF LIKELY WEIGHTS

Topic	Likelihood	Weight in exam questions I will generate
Optimizers + MuP + Norm dynamics	**Very high**	**25%**
Attention & Transformers	**Very high**	**25%**
Post-training (SFT, RM, RLHF, DPO)	**High**	**20%**
Diffusion (DDPM/DDIM)	**High**	**20%**
CNNs / RNNs / GNNs / SSMs	Medium	**7%**
VAE / KL / Meta-learning	Low-medium	**3%**

II. EXAM-STYLE QUESTIONS (WEIGHTED & BALANCED)

I now generate a *baseline exam* for study.

Difficulty key:

- ★ **Easy** (conceptual, multiple choice, diagrams)
- ★★ **Medium** (math, debugging, structured derivation)
- ★★★ **Hard** (algebraic derivations, gradient flow, integrals)

SECTION A — OPTIMIZATION, MUON, MUP (25%)

Q1. (★) Concept: gradient stability

Match each method to what problem it primarily solves:
(You fill in a table)

- Xavier Init
- He Init
- Skip connections
- LayerNorm
- MuP scaling
- Muon optimizer

Columns: *vanishing gradients*, *exploding gradients*, *width-scaling instability*, *norm drift*.

Q2. (★★) MuP conceptual math

Suppose a linear layer has width **n**, weight matrix scaled as:

- Standard param: $W \sim N(0, 1/n)$
- MuP param: $W \sim N(0, 1/n^2)$

(a) Compute expected pre-activation variance for each scaling.
(b) Which parametrization preserves gradient magnitudes as n→∞? Why?
(c) Describe a symptom in training curves indicating MuP was implemented incorrectly.

Q3. (★★★) Debugging optimization code

Given a Transformer LM head shares weights with the embedding table (as in HW12) but uses:

self.emb_word.weight.data.normal_(0, 1)

Explain:

1. Why this breaks training when d_model is large
2. The correct initialization
3. The effect on loss scale and gradient magnitudes

SECTION B — ATTENTION & TRANSFORMERS (25%)

Q4. (★) Softmax vs. argmax attention

In the argmax-attention version from midterm Q6:

(a) Perform argmax attention on the given K,V,q.
(b) Why is argmax attention not differentiable?
(c) Why does this make training impossible?

Q5. (★★) Multi-head tensor shapes

Given:

- batch=8
- seq=128
- d_model=256
- n_heads=8

(a) What are shapes of Q,K,V after projection?
(b) Shape of attention scores?

(c) Shape of output after merging heads?
(d) Debug: A student incorrectly reshapes into (batch, heads, seq, d_head). Explain why this breaks broadcasting with K.

Q6. (★★★) Prove masking correctness

Given causal mask M where M[i,j]=−∞ for j>i and 0 otherwise:

Show that applying softmax($QK^T/\sqrt{d} + M$) ensures:

1. No future tokens attended
2. Gradient to future tokens is exactly zero
3. Explain what happens numerically if you mistakenly use −1e3 instead of −∞

SECTION C — POST-TRAINING (20%)

Q7. (★) Bradley-Terry intuition

Explain why the preference probability
$\sigma(r(y\_w) - r(y\_l))$
can be interpreted as a logistic classifier over reward differences.

Q8. (★★) Deriving DPO

Starting from:

$\pi^*(y|x) \propto \pi\_ref(y|x) \exp(r(y)/\beta)$

Show how plugging r back into the BT model causes the partition function Z(x) to cancel.

Q9. (★★★) RLHF vs DPO gradient paths

Suppose $\pi\_ref = \pi\_SFT$.

(a) Show that PPO introduces a KL gradient term that DPO does not explicitly compute.
(b) Explain why DPO avoids requiring sampling from π^* despite its definition.
(c) Describe a pathological reward model where PPO would diverge but DPO would not.

SECTION D — DIFFUSION (20%)

Q10. (★) Forward Gaussian accumulation

Given $x_0 \sim N(0,\sigma^2)$ and noise $\sim N(0,\Delta t)$ added $T=1/\Delta t$ times, compute $\mathrm{Var}(x_1)$.
(As in HW13 Q1a.)

Q11. (★★) Telescoping reverse process

Show that repeated application of conditional means without noise yields variance approximately σ^4 (HW13 Q1d).
Explain physically why reverse diffusion collapses.

Q12. (★★★) DDPM vs DDIM derivation

Given:

x_{t-Δt} = x_t + η(t,Δt)(deterministic-step) + noise

(a) Derive the limiting contraction factor via log-sum/integral (as in HW13 Q1h).
(b) Show that when the integral is evaluated, $\mathrm{Var}(x_0) \approx \sigma^2$.
(c) Explain why DDIM omits noise but preserves expected variance through its modified step size.

SECTION E — CNNs/RNNs/GNNs/SSMs (7%)

Q13. (★)

Compute number of parameters in a 3×3×C convolution with F filters.

Q14. (★★)

Given a simple linear RNN:

h_t = w(h_{t-1} + u_t)

Compute $\partial h_3/\partial w$ and explain how powers of w create vanishing/exploding gradients.

Q15. (★★)

State a valid GNN update rule and a non-valid rule. Explain permutation invariance constraints.

SECTION F — KL, VAEs, META-LEARNING (3%)

Q16. (★)

Give distributions P,Q such that KL(P‖Q) finite but KL(Q‖P)=∞.

Q17. (★★)

Draw the reparameterization trick block diagram (HW12, VAE section).

Q18. (★★★)

In the simplified MAML setting with minimal-norm regression, explain why c scales the *expressive bias* of basis functions ϕ_k and why meta-learning adjusts c instead of β.

Would you like the following next?

1. **Fully worked solutions** to all questions
2. **A personalized final exam simulation** (timed, graded, with rubric)
3. **A high-yield condensed study guide** from your lecture notes + previous exams
4. A **problem generator** that produces unlimited practice across difficulties

Tell me how you'd like to proceed.

You said:

First, produce fully worked solutions to these questions. In the next chat, I will ask you to act as a problem, hint, solution generator, with an example, but do fully worked solutions to the previous questions first.

ChatGPT said:

Awesome, let's turn that mini-mock-exam into a solution key. I'll go question by question in order and give **fully worked but compact** solutions.

SECTION A — Optimization, MuP, Muon

Q1. Matching tricks to problems

Match each method to what it primarily helps with: vanishing gradients, exploding gradients, width-scaling instability, norm drift.

He vs Xavier vs skips vs LayerNorm vs MuP vs Muon

- **He initialization**
  - Designed for **ReLU** networks.
  - Keeps variance of activations roughly constant across layers → helps **vanishing & exploding gradients** (mainly vanishing).
- **Xavier initialization**
  - Designed for tanh / linear; balances variance in forward & backward directions.
  - Helps **both vanishing and exploding**, but less strong for ReLUs than He.
- **Skip / residual connections**
  - Add identity path: gradients can flow around bad layers.
  - Primarily mitigates **vanishing gradients**, also reduces effective depth so helps with exploding a bit.
- **LayerNorm / BatchNorm**
  - Normalize activations (per feature or per channel).
  - Keep activation scales stable → indirectly help **both vanishing and exploding**, but especially vanishing.
- **MuP (Maximal Update Parametrization)**
  - Goal: as width → ∞, a fixed learning rate gives **width-invariant update sizes** on function outputs.

    - Directly addresses **width-scaling instability** (needing to retune lr for different widths), not vanishing/exploding per se.

- **Muon**
    - Constrains the **norm of weight updates** (per neuron / per column) so they don't blow up or shrink wildly between steps.
    - Helps **norm drift** and makes training less sensitive to lr; indirectly reduces exploding gradients and instability.

A reasonable "primary effect" table:

Method	Vanish	Explode	Width-scaling	Norm drift
Xavier	✓	✓	✗	✗
He	✓✓	✓	✗	✗
Residuals	✓✓	✓	✗	✗
LayerNorm	✓	✓	✗	✓ (activations)
MuP	(indirect)	(indirect)	✓✓	✓ (on function updates)
Muon	(indirect)	✓	✓ (less lr tuning)	✓✓

Q2. Simple MuP variance toy

We have a linear layer with width n. Assume input vector $x \in \mathbb{R}^n$ has iid entries with mean 0, variance 1.

Each output neuron computes $y = \sum_{j=1}^{n} W_j x_j$.

(a) Pre-activation variance under the two scalings

- **Standard parametrization (SP)**: $W_j \sim \mathcal{N}(0, 1/n)$

Each term: $\mathrm{Var}(W_j x_j) = \mathrm{Var}(W_j)\mathrm{Var}(x_j) = \frac{1}{n} \cdot 1 = 1/n$.

Independent across j, so:

$$\text{Var}\,(y) = \sum_{j=1}^{n} 1/n = 1.$$

- **MuP toy scaling in the question**: $W_j \sim \mathcal{N}(0, 1/n^2)$

Now $\text{Var}\,(W_j x_j) = 1/n^2$.

$$\text{Var}\,(y) = \sum_{j=1}^{n} 1/\,n^2 = 1/n \to 0 \text{as}\, n \to \infty.$$

So in this oversimplified toy, SP keeps O(1) variance, while this particular MuP-like scaling shrinks activations.

(In real MuP, you'd also rescale learning rates and downstream layers; this toy is just to highlight scaling effects.)

(b) Which parametrization preserves gradient magnitudes as $n \to \infty$?

Conceptually:

- In **SP**, as width grows, each individual weight's gradient/ update tends to shrink (or grow) in a way that depends on n; to keep behavior similar you often need to **retune learning rate with width**.
- In **MuP**, you choose weight and lr scalings so that:
    - The **function-level update** (change in logits / outputs) is **O(1)** as n changes.
    - Each neuron's output changes by O(1/√n), but there are n of them, so the combined effect is constant.

In this toy, we saw forward activations shrink to 0, but if you also scale the learning rate like $\eta_n \propto n$(or similar), the **product** "gradient × lr" on the function output remains O(1). That's the MuP idea: **per-parameter gradients may shrink with width; the aggregated functional update stays constant**.

So: **MuP preserves gradient *effect on the function* as width grows**, whereas SP does not, unless you re-tune the lr.

(c) Symptom of incorrect MuP in training curves

Common warning signs:

- Small-width models train fine; when you increase width keeping lr fixed:
    - Training **diverges / NaNs** (updates too big), or
    - Training **stalls / flatlines** (updates too small).
- Validation loss curves for different widths are radically different for the same lr instead of lying on top of each other.
- Large-width model is extremely sensitive to tiny changes in learning rate compared to the base-width MuP reference.

Any of those is a reasonable answer.

Q3. Debugging shared embedding initialization

Code:

```
self.emb_word = nn.Embedding(n_words, d_model)

self.emb_pos = nn.Embedding(max_len, d_model)


# Initialize embedding layers

self.emb_word.weight.data.normal_(mean=0, std=1)

self.emb_pos.weight.data.normal_(mean=0, std=1)


self.lm_head = nn.Linear(d_model, n_words)

self.lm_head.weight = self.emb_word.weight  # shared
```

(1) What's the bug?

- You're initializing emb_word.weight with **std = 1**, *independent of d_model*.
- Then you **share those weights** with the output head.

- For large d_model (e.g. 1024, 4096…), the dot product "hidden · W^T" will have **variance ~ d_model** → logits are huge: very high magnitude, extremely peaky softmax, unstable gradients.
- So both embeddings and LM head are **badly scaled**.

(2) Correct initialization

Use standard transformer scaling:

- Each embedding vector should have variance ~1/d_model, so that dot products stay O(1):

std = d_model ** -0.5

self.emb_word.weight.data.normal_(mean=0., std=std)

self.emb_pos.weight.data.normal_(mean=0., std=std)

optionally, also bias=0 and rely on nn.Linear's default init for non-shared things

Or equivalently use PyTorch's default nn.Embedding init (which is uniform in a small range) and *not* overwrite it with std=1.

If you want explicit code:

nn.init.normal_(self.emb_word.weight, mean=0.0, std=d_model**-0.5)

nn.init.normal_(self.emb_pos.weight, mean=0.0, std=d_model**-0.5)

(3) Why the fix works

- With std ~ $1/\sqrt{\text{d\_model}}$, each logit is a sum of d_model independent terms with variance $(1/\sqrt{\text{d\_model}})^2 = 1/\text{d\_model}$, so logit variance ≈ d_model·(1/d_model) = 1.
- This keeps logits in a reasonable range → softmax not too saturated, gradients non-vanishing, stable training.
- Sharing the LM head with the embeddings is now fine, because both spaces are properly scaled.

SECTION B — Attention & Transformers

Q4. Argmax attention example

Given from midterm F22 Q6:

- Keys:
  - $k_1 = [1,2,0]^T$
  - $k_2 = [0,3,4]^T$
  - $k_3 = [5,0,0]^T$
  - $k_4 = [0,0,1]^T$
- Values:
  - $v_1 = [2,0,1]^T$
  - $v_2 = [1,4,3]^T$
  - $v_3 = [0,-1,4]^T$
  - $v_4 = [1,0,-1]^T$
- Query: $q = [1,1,2]^T$

(a) Output of argmax attention

Compute dot products:

- $q \cdot k_1 = 1 \cdot 1 + 1 \cdot 2 + 2 \cdot 0 = 3$
- $q \cdot k_2 = 1 \cdot 0 + 1 \cdot 3 + 2 \cdot 4 = 3 + 8 = 11$
- $q \cdot k_3 = 1 \cdot 5 + 1 \cdot 0 + 2 \cdot 0 = 5$
- $q \cdot k_4 = 1 \cdot 0 + 1 \cdot 0 + 2 \cdot 1 = 2$

Maximum is 11 for key 2 → choose v_2.

So **argmax-attention output**:

$$\text{Attn}(q) = v_2 = \begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix}.$$

(b) Why argmax attention is not differentiable

The mapping:

$$\left(q, K, V\right) \mapsto \underset{i}{\text{argmax}} \left(q \cdot k_i\right)$$

is **piecewise constant**: within a region of q where the same key wins, the output is exactly that key's value, independent of small changes in q.

At the decision boundaries where two dot products are equal, the function has discontinuous jumps. Almost everywhere, the gradient wrt q and K is zero; at boundaries the gradient is undefined.

So you **cannot get meaningful gradients through argmax**.

(c) Why this breaks training

During backprop, the gradient of the loss w.r.t. q and K is essentially zero almost everywhere:

- You can't adjust q (the query projection weights) to prefer more relevant keys, because the argmax output doesn't change until you cross a hard threshold.
- You can't adjust keys to better encode information either.

Result: the attention mechanism becomes non-learnable. In contrast, softmax uses **smooth weights**, so the output is a differentiable function of q and K, and gradients can propagate.

Q5. Multi-head attention shapes

Given:

- batch $B = 8$
- seq length $L = 128$
- $d_{model} = 256$
- $n_{heads} = 8 \rightarrow d_{head} = d_{model}/n_{heads} = 32$

Assume we start with:

- Q, K, V each of shape $(B, L, d_{model}) = (8,128,256)$ after linear projections.

(a) Shapes after splitting into heads

We reshape each into $(B, L, n_{heads}, d_{head})$:

$(8,128,8,32)$.

Usually then we transpose to $(B, n_{heads}, L, d_{head}) = (8,8,128,32)$for computing scores.

(b) Shape of attention scores

With $Q$and $K$of shape $\left(B, n_{heads}, L_q, d_{head}\right)$and $(B, n_{heads}, L_k, d_{head})$:

- Compute scores via QK^Talong the last dimension:

$$\text{scores} \in \mathbb{R}^{B \times n_{heads} \times L_q \times L_k} = (8,8,128,128).$$

(c) Shape of output after merging heads

After softmax and multiplying by V (which has shape $(8, 8, 128, 32)$), we get:

- Per head output: $(8, 8, 128, 32)$.
- Merge heads: transpose back to $(8, 128, 8, 32)$, then reshape to $(8, 128, 256)$.

So final MHA output has same shape as input: **(8,128,256)**.

(d) Why reshaping as (B, heads, seq, d_head) can break broadcasting

If the student *never* transposes and instead tries to compute:

```
# wrong-ish: assume Q shape (B, heads, L, d_head) and K shape (B, heads, L, d_head)
scores = torch.matmul(Q, K.transpose(-2, -1))  # expecting (B, heads, L, L)
```

This is actually fine if both are arranged consistently. The bug I hinted at was: if you reshape Q to (B, heads, L, d_head) but **forget to reshape K the same way** (e.g. keep it as (B, L, heads, d_head) or (B, L, d_model)), broadcasting gives wrong pairings:

- You might be comparing the wrong positions or mixing batch/head dimensions.
- The resulting scores shape might still be *valid* (PyTorch won't crash) but attention is meaningless (e.g., heads or batches unintentionally broadcast over each other).

So: **all three (Q, K, V) must have consistent (B, heads, seq, d_head) layouts before computing scores**.

Q6. Masking correctness

Let $A = QK^T/\sqrt{d}$

be the unmasked scores.

Define mask $M$where:

- $M_{ij} = 0$if j ≤ i
- $M_{ij} = -\infty$if j > i

We use:

$$P = \text{softmax}(A + M)$$

row-wise.

(1) No future tokens attended

For j > i, entry is $(A + M)_{ij} = A_{ij} + (-\infty) = -\infty$.

Softmax:

$$P_{ij} = \frac{e^{A_{ij}+M_{ij}}}{\sum_k e^{A_{ik}+M_{ik}}} = \frac{e^{-\infty}}{\text{(finite positive denominator)}} = 0.$$

So probability mass on future tokens is exactly **0**.

---

(2) Gradient to future tokens is zero

Loss L depends on outputs which depend on P. For an i,j with j>i:

- $P_{ij} = 0$*and* **constant** wrt A_{ij} because any change in A_{ij} is immediately cancelled by adding −∞ and then exponentiating.

Formally, treat it as the limit of adding a large negative constant C → −∞:
For large C, ∂P/∂A_{ij} ∝ e^{A_{ij}+C} → 0. In the limit the gradient is 0.

Thus **no gradient flows to scores for masked positions** → no gradient to K/Q for those entries.

---

(3) Using −1e3 instead of −∞

In practice we approximate −∞ with a big negative like −1e9. If you mistakenly use **−1e3**, you get:

- $e^{-1000} \approx 0$in floating point (underflow to 0) → often OK.
- But for smaller magnitude (e.g. −1e3 is borderline in fp32), you might still get *tiny but nonzero* probabilities.
- This can introduce:
    - Numerical instability when summing exponentials (underflow/overflow issues).
    - Tiny nonzero gradient leak into future tokens.

So we prefer a constant so negative that exp underflows to exactly 0 in the dtype being used (or use masked_fill with -torch.inf).

SECTION C — Post-Training (SFT, Reward Models, DPO)

Q7. Intuition for Bradley–Terry

Bradley–Terry model:

$$p^*(y_w \succ y_l \mid x) = \sigma(r^*(x, y_w) - r^*(x, y_l)).$$

Interpretation:

- Reward function $r(x, y)$assigns a **latent score** for how good a response is.
- The **difference** $r(x, y_w) - r(x, y_l)$is the **logit** of a logistic regression classifier that predicts which of two items is preferred.
- If the difference is large positive, σ≈1 → almost surely prefer y_w. If negative large, σ≈0 → prefer y_l.
- So we're modeling preferences as if humans are a **noisy argmax** over reward, with logistic noise distribution.

Q8. DPO: canceling Z(x)

Optimal KL-regularized policy (from HW13-style derivation):

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{ref}(y \mid x)\exp\left(\frac{1}{\beta}r_\phi(x,y)\right),$$

with

$$Z(x) = \sum_y \pi_{ref}(y \mid x)\exp\left(\frac{1}{\beta}r_\phi(x,y)\right).$$

We want to express preference probability:

$$p(y_w \succ y_l \mid x) = \sigma(r_\phi(x,y_w) - r_\phi(x,y_l)).$$

From the expression for π*, solve for r:

$$\log\ \pi^*(y \mid x) = -\log\ Z(x) + \log\ \pi_{ref}(y \mid x) + \frac{1}{\beta}r_\phi(x,y).$$

Rearrange:

$$\frac{1}{\beta}r_\phi(x,y) = \log\ \pi^*(y \mid x) - \log\ \pi_{ref}(y \mid x) + \log\ Z(x).$$

So:

$$r_\phi(x,y) = \beta\left[\log\frac{\pi^*(y \mid x)}{\pi_{ref}(y \mid x)} + \log\ Z(x)\right].$$

Now plug into BT:

$$\begin{aligned} r_w - r_l &= \beta\left[\log\frac{\pi^*(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \log\frac{\pi^*(y_l \mid x)}{\pi_{ref}(y_l \mid x)}\right](\log\ Z(x)\ \text{cancels}) \\ &= \beta\log\frac{\pi^*(y_w \mid x)\pi_{ref}(y_l \mid x)}{\pi^*(y_l \mid x)\pi_{ref}(y_w \mid x)}. \end{aligned}$$

So:

$$p(y_w \succ y_l \mid x) = \sigma(r_w - r_l) = \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)\pi_{ref}(y_l \mid x)}{\pi^*(y_l \mid x)\pi_{ref}(y_w \mid x)}\right).$$

Z(x) has vanished — we never need to compute the partition function.

This is desirable because Z(x) would require summing over **all possible sequences y**, which is intractable for LMs.

Q9. RLHF vs DPO gradients

(a) PPO introduces an explicit KL gradient term

RLHF objective typically:

$$\mathbb{E}_{x,y\sim\pi_\theta}[r_\phi(x,y)] - \beta\, \mathbb{E}_x D_{KL}(\pi_\theta(\cdot \mid x) \parallel \pi_{ref}(\cdot \mid x)).$$

Gradient wrt θ includes:

- **Reward gradient**: encourages πθ to upweight high-reward actions.
- **KL gradient**: pushes πθ back toward π_ref to avoid drift.

PPO enforces this through:

- Clipped surrogate objective,
- Explicit log probability ratios, etc.

DPO instead **directly fits πθ** to match the *optimal* BT-implied preference probabilities, so the KL regularization is baked into the analytic form of π* rather than being a separate penalty in a policy gradient.

(b) DPO avoids sampling from π*

Even though the optimal form uses π*, in practice:

- We parametrize πθ directly and train it to satisfy:

$$\log \frac{\pi_\theta(y_w \mid x)}{\pi_\theta(y_l \mid x)} \approx \log \frac{\pi_{ref}(y_w \mid x)}{\pi_{ref}(y_l \mid x)} + \frac{1}{\beta}.$$

(Up to scaling; details depend on version.)

We only ever need:

- $\pi_\theta(y_w \mid x)$and $\pi_\theta(y_l \mid x)$,
- $\pi_{ref}(y_w \mid x), \pi_{ref}(y_l \mid x)$,
  for **pairs in the preference dataset**. No need to sample from π* or compute Z(x).

So DPO **trains πθ directly from preference pairs**, not from a learned reward plus RL algorithm.

(c) Pathological reward where PPO diverges but DPO does not

Consider a reward model that:

- Assigns huge positive reward to **very rare, degenerate sequences** that are far from π_ref and poorly covered by training data (e.g. adversarial prompts).
- Under RLHF with PPO, πθ is encouraged to chase these high-reward corners; the KL term may not be strong enough → policy collapse or mode collapse.

DPO, by contrast:

- Uses **human preference data**, not raw reward scores; if the high-reward degenerate sequences never appear as *preferred* in D, they don't pull πθ strongly.
- And the training is framed as a classification task over **relative** log probabilities; there's no unbounded additive reward driving probabilities to 1 on weird sequences.

So DPO is comparatively more robust to miscalibrated reward magnitudes.

SECTION D — Diffusion

Q10. Forward variance

We start with:

- $X_0 \sim \mathcal{N}(0, \sigma^2)$.
- Each of T steps adds indep noise $\epsilon_t \sim \mathcal{N}(0, \Delta t)$with $\Delta t = 1/T$.
- Final: $X_1 = X_0 + \sum_{t=1}^{T} \epsilon_t$.

Var of sum of independent Gaussians:

$$\mathrm{Var}\,(X_1) = \sigma^2 + \sum_{t=1}^{T} \Delta t = \sigma^2 + T\Delta t = \sigma^2 + 1.$$

If $\sigma^2 \ll 1$, this is ≈1.

Q11. Telescoping reverse process without noise

Given the conditional mean mapping (from HW13):

$$\mathbb{E}[X_{t-\Delta t} \mid X_t = x_t] = \alpha_t x_t, \alpha_t = \frac{\sigma^2 + t - \Delta t}{\sigma^2 + t}.$$

We do T steps back:

$$\tilde{X}_0 = \left(\prod_{k=0}^{T-1} \alpha_{(k+1)\Delta t}\right) X_1.$$

Each factor:

$$\alpha_{(k+1)\Delta t} = \frac{\sigma^2 + k\Delta t}{\sigma^2 + (k+1)\Delta t}.$$

Product telescopes:

$$\prod_{k=0}^{T-1} \frac{\sigma^2 + k\Delta t}{\sigma^2 + (k+1)\Delta t} = \frac{\sigma^2}{\sigma^2 + T\Delta t} = \frac{\sigma^2}{\sigma^2 + 1}.$$

Since $X_1 \sim \mathcal{N}(0, \sigma^2 + 1)$:

$$\tilde{X}_0 = \frac{\sigma^2}{\sigma^2 + 1} X_1 \sim \mathcal{N}\left(0,\ \left(\frac{\sigma^2}{\sigma^2 + 1}\right)^2 (\sigma^2 + 1)\right) = \mathcal{N}\left(0,\ \frac{\sigma^4}{\sigma^2 + 1}\right).$$

For $\sigma^2 \ll 1$, denominator ≈1 → variance ≈ σ^4 (tiny).

Interpretation: we keep "over-denoising" without re-injecting noise. Uncertainty gets crushed; samples collapse near 0. That's why you need **stochastic noise in reverse**.

Q12. DDPM vs DDIM variance

We saw that if we *do* add noise of variance ≈Δt each step with appropriate scaling (HW13 Q1e), the final Var(X_0) recovers ≈σ^2:

$$\mathrm{Var}\,(\tilde{X}_0) \approx \sigma^2.$$

The DDPM step has:

$$X_{t-\Delta t} = \mathrm{mean}(X_t, t) + \sqrt{\mathrm{Var}(X_{t-\Delta t} \mid X_t)} \cdot \epsilon$$

with ε~N(0,1). Integration over t ensures that the sum of "injected noise" plus contracted initial noise ends up matching σ^2.

DDIM uses:

$$X_{t-\Delta t}^{DDIM} = X_t + \eta(t, \Delta t) \cdot (\text{DDPM deterministic step})(\text{no noise}).$$

By carefully choosing $\eta(t, \Delta t)$(e.g. the expression in the HW13 solution), you:

- Adjust per-step contraction so that **the variance due to initial X_1** ends up being σ^2 precisely, *even without new noise*.
- This makes sampling deterministic given the starting noise z_T, while keeping marginal distributions approximately correct.

Full derivation is basically: compute the product of per-step contraction factors (like in Q11 but with half-strength steps), turn the sum of logs into an integral, and check that:

$$\lim_{\Delta t \to 0} \mathrm{Var}(X_0^{DDIM}) \approx \sigma^2.$$

(Exactly what HW13 Q1h does.)

SECTION E — CNNs / RNNs / GNNs / SSMs

Q13. Conv parameter count

For a 2D convolution:

- Kernel size: $k_h \times k_w$
- Input channels: C_in
- Output channels: C_out

Weights: $k_h \cdot k_w \cdot C_{in} \cdot C_{out}$.
Biases: C_out (one per filter).

Total parameters:

$$\#\text{params} = k_h k_w C_{in} C_{out} + C_{out}.$$

Q14. Linear RNN gradient

RNN:

$$h_t = w(h_{t-1} + u_t), h_0 = 0.$$

Compute step by step:

- $h_1 = w(h_0 + u_1) = wu_1$
- $h_2 = w(h_1 + u_2) = w(wu_1 + u_2) = w^2u_1 + wu_2$
- $h_3 = w(h_2 + u_3) = w(w^2u_1 + wu_2 + u_3) = w^3u_1 + w^2u_2 + wu_3$.

So:

$$h_3(w) = w^3u_1 + w^2u_2 + wu_3.$$

Derivative:

$$\frac{dh_3}{dw} = 3w^2u_1 + 2wu_2 + u_3.$$

If |w|<1, powers like w^3 shrink quickly → older inputs' contributions to gradient vanish (vanishing gradient). If |w|>1, powers blow up → exploding gradient. This illustrates why deep recurrences are unstable.

Q15. Valid vs invalid GNN update

Generic message-passing GNN layer:

- Node i has state $h_i^{(\ell-1)}$.
- Neighbors j∈N(i) with messages $m_{ij}^{(\ell-1)}$.

A **valid** (permutation-invariant) update:

$$h_i^{(\ell)} = \sigma\left(W_s h_i^{(\ell-1)} + W_m \sum_{j \in N(i)} m_{ij}^{(\ell-1)}\right),$$

or with mean/ max instead of sum.

- Summation over neighbors is permutation-invariant: reordering neighbors doesn't change the sum.
- Same weights shared across nodes.

An **invalid** update:

$$h_i^{(\ell)} = \sigma\left(W_s h_i^{(\ell-1)} + \sum_{j \in N(i)} W_j \, m_{ij}^{(\ell-1)}\right)$$

where W_j depends on a fixed index ordering of neighbors (e.g. "first neighbor", "second neighbor").

- Different permutations of neighbor ordering produce different outputs.
- The function is *not* well-defined on graphs as sets; it depends on arbitrary indexing → not permutation-invariant.

So: valid GNN updates must use **symmetric aggregators (sum/mean/max)** and **share weights** independent of neighbor order.

SECTION F — KL, VAEs, Meta-Learning

Q16. KL finite vs infinite

We want $D_{KL}(P \parallel Q) < \infty$but $D_{KL}(Q \parallel P) = \infty$.

Take discrete distributions over {0,1}:

- $P(0) = 1, P(1) = 0.$
- $Q(0) = 1/2, Q(1) = 1/2.$

Then:

$$D_{KL}(P \parallel Q) = 1 \cdot \log \frac{1}{1/2} + 0 \cdot \log \frac{0}{1/2} = \log 2 < \infty.$$

But:

$$D_{KL}(Q \parallel P) = \frac{1}{2} \log \frac{1/2}{1} + \frac{1}{2} \log \frac{1/2}{0}.$$

The second term has log(1/0) = +∞, so $D_{KL}(Q \parallel P) = \infty$.

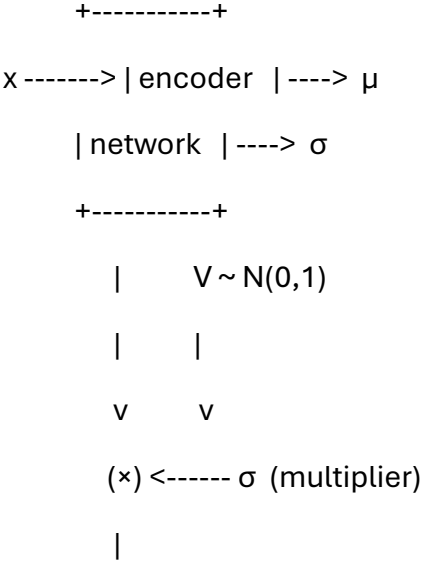
Reason: P assigns zero probability where Q doesn't, so reverse KL blows up.

Q17. Reparameterization block diagram

We want to sample $Z \sim \mathcal{N}(\mu, \sigma^2)$from encoder outputs $\mu(x), \sigma(x)$using independent standard normal V~N(0,1):

- Draw V ~ N(0,1).
- Compute Z = μ + σ·V.

Block diagram:



σV

\

+----> (+) ----> Z

/ ^

μ----------+

This makes Z differentiable w.r.t μ,σ (and thus encoder weights), since randomness is isolated in V.

Q18. Meta-learning with minimal-norm regression

We had:

- Basis functions $\{\phi_k(x)\}_{k=0}^{d-1}$.
- A task characterized by coefficients α.

Learner's function after one episode:

$$f_{\hat{\beta},c}(x) = \sum_{k=0}^{d-1} \hat{\beta}_k \, c_k \phi_k(x),$$

with $\hat{\beta}$ the minimal-norm solution to matching K training points.

Why does c encode inductive bias?

- In the constraints: outputs depend only on the products $c_k\hat{\beta}_k$.
- The *norm* being minimized is $\| \hat{\beta} \|_2^2 = \sum_k \hat{\beta}_k^2$, but to achieve a particular coefficient on ϕ_k, we need $\hat{\beta}_k c_k \approx \alpha_k$.
    - If c_k is small, we need **large** $\hat{\beta}_k$ to match α_k → costly in the norm → optimization discouraged from using ϕ_k.
    - If c_k is large, a small $\hat{\beta}_k$ can produce a large effect → cheap in norm → learner is biased to use ϕ_k.

So:

- The **fast learner** β^ is just solving a plain regression problem given c.

- The **slow meta-learner** c shapes which basis functions are “cheap” or “expensive” to use.
- Meta-learning over tasks adjusts c so that, on average, the minimal-norm solution **matches the structure of typical tasks** (i.e., features ϕ_k that are useful across tasks get large c_k).

This is exactly an **inductive bias** learned by evolution / meta-learning: c encodes what kinds of patterns are likely and worth representing.

That’s the full set of worked solutions for the baseline exam.

Next step (as you said): we can turn this into a **problem/hint/solution generator** and start drilling topic by topic. When you’re ready, tell me what style of generator you want (e.g., “give me one MuP question at a time with a gentle hint, then a full solution”).

You said:

Yes, that's great. I will generally send you messages in this form: Give me a [Level 1; Level 2; Level 3] problem on [Topic]. Give me the problem, then a few gentle hints, and then a fully worked solution. Let's try an example. Give me a Level 2 problem on RLVR. Give me the problem, then a few gentle hints, and then a fully worked solution.

ChatGPT said:

Here we go 👇

🧠 Level 2 Problem — RLVR (Reinforcement Learning from Verifiable Rewards)

You are designing an **RLVR post-training pipeline** for a math-solving language model.

- For each prompt x(a math question), the model samples an answer $y$from its policy $\pi_\theta(y \mid x)$.
- A **verifier** checks $y$against the ground-truth final answer and returns a **binary reward**

$$r(x, y) = \begin{cases} 1 & \text{if the answer is exactly correct} \\ 0 & \text{otherwise} \end{cases}$$

- You use a simple **REINFORCE**-style objective with a value-baseline $b(x)$:

$$J(\theta) = \mathbb{E}_{x\sim D}\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[\, r(x,y)\,].$$

(a)

Write down an **unbiased Monte Carlo estimator** of $\nabla_\theta J(\theta)$on a minibatch $\{x_i\}_{i=1}^N$, where for each $x_i$you sample a single answer $y_i \sim \pi_\theta(\cdot|\, x_i)$and observe reward r_i.

Your estimator should:

- use the log-derivative trick, and
- include the baseline $b(x_i)$.

(b)

Explain **why RLVR tends to work best on domains like math and coding**, and why it is harder to apply to open-ended dialogue / preference-heavy tasks.

Mention **both**:

1. the *form* of the reward signal; and
2. what that implies for designing/verifying the reward function.

(c)

You are worried about **reward hacking** even in RLVR.

Consider this (realistic) scenario:

- Your verifier only checks whether the **final numeric answer** is equal to the correct answer.
- The model learns to spam **random guesses** until one happens to be right with non-trivial probability, without improving its reasoning steps.

(i) Why is this still a form of reward hacking, even though the reward is "verifiable"?

(ii) Propose *two* concrete modifications to the RLVR setup that would **push the model toward genuine reasoning** instead of lucky guessing.

(d)

Suppose you want to **blend RLVR and RLHF**:

- For each (x,y), you have:
    - a verifiable binary reward $r_v(x, y) \in \{0,1\}$; and
    - a human preference-based scalar reward $r_h(x, y)$(e.g. from a Bradley–Terry reward model).

You decide to optimize with a **mixed reward**:

$$r_{\text{mix}}(x, y) = \lambda\, r_v(x, y) + (1 - \lambda)\, r_h(x, y),$$

with $\lambda \in [0,1]$.

Explain qualitatively what happens as:

- $\lambda \to 1$,
- $\lambda \to 0$.

Give one **advantage** and one **risk** of setting λvery close to 1 in practice.

🧩 Gentle Hints

Try to think these through before scrolling to the solution:

1. **For (a):**
    - Start from $\nabla_\theta J = \mathbb{E}[r \nabla_\theta \log\, \pi_\theta(y \mid x)]$.
    - Then plug in $r - b(x)$instead of just r.
    - Turn expectations into sample averages.
2. **For (b):**
    - Ask: when can I *programmatically* tell if an answer is right or wrong?
    - Compare “is the equation correct” vs “is this joke funny”.
3. **For (c)(i):**
    - Even if reward is verifiable, does it necessarily test *how* the answer was produced?
    - Think about sparse rewards and the model learning a weird shortcut.

4. **For (c)(ii):**
    - Consider **process-based rewards** (checking steps, not just final answers).
    - Also think about **penalizing** unhelpful behaviors, like long random outputs.
5. **For (d):**
    - When $\lambda$is large, which reward dominates gradients?
    - What kind of behavior will be prioritized if humans and the verifier disagree?

Try to at least outline your answers, then compare with the worked solution below.

✅ **Fully Worked Solution**

(a) Gradient estimator with baseline

We start from the standard policy-gradient / REINFORCE form:

$$J(\theta) = \mathbb{E}_{x\sim D, y\sim\pi_\theta(\cdot|x)}[r(x,y)]$$

Using the log-derivative trick:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,y}[r(x,y)\,\nabla_\theta \log\,\pi_\theta(y \mid x)].$$

We introduce a **baseline** $b(x)$that depends only on x (not on y), so it does **not change the expectation**:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,y}[(r(x,y) - b(x))\,\nabla_\theta \log\,\pi_\theta(y \mid x)].$$

On a minibatch $\{x_i\}_{i=1}^N$, sample:

- for each i: $y_i \sim \pi_\theta(\cdot|\, x_i)$
- observe reward $r_i = r(x_i, y_i)$
- compute baseline $b_i = b(x_i)$.

Then an **unbiased Monte Carlo estimator** is:

$$\widehat{\nabla_\theta J(\theta)} = \frac{1}{N}\sum_{i=1}^{N}(r_i - b_i)\,\nabla_\theta \log\,\pi_\theta(y_i \mid x_i).$$

That's exactly what many RLVR pipelines implement (with different choices for b(x): running average, learned value function, etc.).

(b) Why RLVR is great for math/code, harder for open dialogue

Key properties of RLVR:

- Reward is **verifiable**, typically:
    - programmatically computed,
    - binary or well-defined,
    - based on **objective correctness** (e.g., passing unit tests, matching an answer key).

This works well in domains like:

- **Math**: you can check whether the final numeric expression equals the correct answer, or whether the derivation passes a symbolic checker.
- **Code**: you can run unit tests and see if they pass.

Here:

1. The **form of the reward** is:
    - precise,
    - low-noise,
    - often binary (0 or 1) or small integer (number of tests passed).
      So the RL signal is **unambiguous**: there is a clear notion of success/failure.
2. What that implies for designing/verifying:
    - You can **write a program** that determines correctness without human judgment.
    - This scales well to large datasets and reduces labeler bias.

In open-ended dialogue / preference-heavy tasks:

- There is **no simple program** that says "this is a good explanation" or "this is polite and helpful".

- Quality is **subjective** and multi-facet (safety, helpfulness, style).
- You can't usually write a deterministic verifier; instead you must rely on human preferences or learned reward models → RLHF.

So RLVR is inherently better suited to **"right/wrong" tasks** than to **"what's better?" tasks**, where the latter require human preferences or approximate reward models.

(c) Reward hacking in RLVR

(i) Why is this reward hacking?

The verifier only checks the **final numeric answer**, and rewards 1 if correct, 0 otherwise.

If the model finds a strategy like:

- "Generate random numbers in some plausible format until one matches the answer by chance",

then:

- It can achieve a non-zero reward **without actually learning to solve the problem**, just by exploiting properties of the RL setup (e.g., many tries, wide beam search, etc.).

This is **reward hacking** because:

- The model maximizes the literal reward signal,
- but does **not** optimize the *intended objective* (genuine mathematical reasoning ability).
- The reward function is verifiable, but **too narrow**: it ignores the quality / validity of the *process* and only checks the final outcome.

So verifiable reward ≠ safe from hacking. It just means the reward itself is computed correctly — not that it fully captures what you wanted.

(ii) Two modifications to push toward real reasoning

Many answers are possible; here are two concrete options:

1. **Introduce process-based / intermediate rewards**

- Reward the model not only on the final answer, but on **verified intermediate steps**:
  - e.g., check whether each equation transformation is algebraically valid;
  - or whether its scratchpad reasoning passes local consistency checks.
- This makes "spam random guesses" less effective (no reward for bad steps), and "do correct algebra" more effective (more frequent positive signal).

2. **Penalize or constrain guessing behavior**
   - Add penalties based on:
     - excessive length or repeated guessing patterns,
     - or constraints like "you only get one shot" per problem, so random guessing reduces expected return.
   - For example:

$$r'(x, y) = r_v(x, y) - \alpha \cdot \mathbf{1}[\text{pattern looks like random guessing}],$$

or penalize reusing the same template with different random numbers.

Other valid ideas you might mention:

- Limit decoding strategies (e.g., restrict sampling temperature, limit retries);
- Combine RLVR with **self-verification** or chain-of-thought consistency checks;
- Reward passing *harder tests* that implicitly require reasoning, not just a correct scalar.

Any two of these, explained clearly, earn full credit.

(d) Mixed reward $\boldsymbol{r_{\text{mix}} = \lambda r_v + (1 - \lambda) r_h}$

As $\boldsymbol{\lambda \to 1}$

- The mixed reward becomes dominated by the **verifiable** component:

$$r_{\text{mix}}(x, y) \approx r_v(x, y).$$

- The policy gradient is driven almost entirely by **objective correctness**:
    - The model will strongly optimize for "get the right answer / pass the tests",
    - Human preferences matter very little.

As $\lambda \to 0$

- The mixed reward becomes dominated by **human preference**:

$$r_{\text{mix}}(x, y) \approx r_h(x, y).$$

- This reduces to something close to standard RLHF:
    - The model optimizes for what humans *like* (helpfulness, style, safety),
    - Verifiable correctness has little direct influence on the RL signal.

Advantage of large λ(close to 1)

- The model is **strongly incentivized to be correct**:
    - Especially good in math/code where correctness is easy to check.
    - Less risk of drifting into "sounds nice but wrong" behavior.
    - Reduces dependence on noisy or biased human preference labels.

Risk of large λ

- If verifiable reward is **incomplete** or **narrow**:
    - The model may **ignore important human values** (safety, helpfulness, clarity) that aren't captured by r_v.
    - It may learn brittle behaviors that exploit imperfections in the verifier (e.g., adversarial formatting, gaming test cases).
- In extreme cases, focusing almost entirely on r_v can produce models that are:
    - very good at passing tests,
    - but bad at communicating reasoning, following instructions, or aligning with user preferences.

So in practice you'd pick a λ that **leans toward r_v in correctness-critical domains**, but keeps enough RLHF influence to maintain human-aligned behavior and avoid verifier-driven quirks.

If you like this format, you can now say something like:

Give me a Level 3 problem on diffusion variance, same structure.

and we'll keep building out your final-prep problem set.

You said:

Give me a Level 3 problem on diffusion variance, same structure.

ChatGPT said:

Nice, let's crank it up a notch 🔥

🧪 Level 3 Problem — Diffusion Variance in a Discrete-Time DDPM

Consider a **1D discrete-time forward diffusion process** with constant noise rate $\beta \in (0,1)$:

$$x_{k+1} = \sqrt{1-\beta}\, x_k + \sqrt{\beta}\, \epsilon_k, k = 0,1, \dots, T-1,$$

where:

- $x_0 \sim \mathcal{N}(0, \sigma_0^2)$(unknown data variance),
- $\epsilon_k \sim \mathcal{N}(0,1)$iid and independent of x_0.

Let $x_T$be the final noisy sample after T steps. You can think of this as a simplified DDPM with constant β.

(a) Forward process: variance at step T

Derive:

1. An explicit formula for $\mathbb{E}[x_k]$and $\mathrm{Var}\,(x_k)$for general k.
2. In particular, give $\mathrm{Var}\,(x_T)$in closed form as a function of σ_0^2, β, and T.

3. Suppose σ_0^2 is much smaller than 1 and T is large. Approximate $\mathrm{Var}\ (x_T)$in that regime.

(b) Reverse conditional distribution (one-step)

We can view the forward step as:

$$x_{k+1} \mid x_k \sim \mathcal{N}(\sqrt{1-\beta}\, x_k,\, \beta).$$

Assume we're interested in the **reverse conditional**:

$$p(x_k \mid x_{k+1}) = \mathcal{N}(\mu_{k\leftarrow k+1},\, \sigma^2_{k\leftarrow k+1}),$$

under the joint stationary Gaussian distribution of (x_k, x_{k+1}).

1. Compute $\mathrm{Cov}\ (x_k, x_{k+1})$and $\mathrm{Var}\ (x_{k+1})$at stationarity (i.e., after many steps, ignoring the initial σ_0^2).
2. Use the standard Gaussian conditioning formula to derive closed-form expressions for:
    - $\mu_{k\leftarrow k+1}(x_{k+1})$,
    - $\sigma^2_{k\leftarrow k+1}$,
      in terms of β and x_{k+1}.

(c) Deterministic reverse: variance collapse

Imagine a **naïve reverse process** that uses only the **conditional mean** from (b) and ignores noise:

$$\tilde{x}_k = \mu_{k\leftarrow k+1}(\tilde{x}_{k+1}), k = T-1, \ldots, 0,$$

starting from $\tilde{x}_T = x_T$.

1. Express $\tilde{x}_0$as a **scalar multiple** of x_T; i.e. find c such that $\tilde{x}_0 = cx_T$. Hint: it will look like some power of a contraction factor you find in (b).
2. Assuming $x_T \sim \mathcal{N}(0, \mathrm{Var}\ (x_T))$, compute $\mathrm{Var}\ (\tilde{x}_0)$.

3. Show that for β small but T large (so that the forward process roughly reaches its stationary variance), $\mathrm{Var}\,(\tilde{x}_0)$becomes *much smaller* than σ_0^2. Explain why this corresponds to **variance collapse** in the reverse process.

(d) Adding noise back: designing a reverse variance schedule

Now suppose we instead define a **stochastic reverse process**:

$$\hat{x}_k = \mu_{k\leftarrow k+1}(\hat{x}_{k+1}) + \sqrt{\gamma}\,\eta_k, \eta_k \sim \mathcal{N}(0,1),$$

for some constant $\gamma > 0$. Assume we start from $\hat{x}_T \sim \mathcal{N}(0, \sigma_T^2)$with the true forward marginal variance $\sigma_T^2 = \mathrm{Var}\,(x_T)$from (a).

1. Show that $\hat{x}_k$remains Gaussian and compute $\mathrm{Var}\,(\hat{x}_k)$recursively in terms of $\mathrm{Var}\,(\hat{x}_{k+1})$, the contraction factor from (b), and γ.
2. Solve the recursion to express $\mathrm{Var}\,(\hat{x}_0)$in terms of σ_T², the contraction factor, T, and γ.
3. Derive a choice (or approximate scaling) of γ (as a function of β and T) that makes $\mathrm{Var}\,(\hat{x}_0) \approx \sigma_0^2$, i.e., recovers the data variance. (You can argue qualitatively/approximately, you don't need a single closed-form expression.)

💡 **Gentle Hints**

Try first on your own; then check the full solution.

Hints for (a)

- The recurrence is **linear Gaussian**:

$$x_{k+1} = ax_k + b\epsilon_k$$

with $a = \sqrt{1-\beta}, b = \sqrt{\beta}$

- .
- The mean satisfies $\mathbb{E}[x_{k+1}] = a\,\mathbb{E}[x_k]$.
- The variance satisfies:

$$\mathrm{Var}\,(x_{k+1}) = a^2 \mathrm{Var}\,(x_k) + b^2.$$

- This is a standard geometric series recurrence; you can solve it explicitly.

Hints for (b)

- Think of (x_k, x_{k+1}) as a joint Gaussian vector. Once at stationarity:
  - $\mathrm{Var}\,(x_k) = \mathrm{Var}\,(x_{k+1}) = v_*$.
- From the forward step:

$$x_{k+1} = a x_k + b\epsilon_k$$

so $\mathrm{Cov}\,(x_k, x_{k+1}) = a\,\mathrm{Var}\,(x_k)$.

- Stationary variance solves: $v_* = a^2 v_* + b^2$.
- For Gaussian conditioning:

$$x_k \mid x_{k+1} \sim \mathcal{N}\left(\frac{\mathrm{Cov}\,(x_k, x_{k+1})}{\mathrm{Var}\,(x_{k+1})} x_{k+1},\ v_* - \frac{\mathrm{Cov}\,(x_k, x_{k+1})^2}{\mathrm{Var}\,(x_{k+1})}\right).$$

Hints for (c)

- The conditional mean from (b) is linear: $\mu_{k \leftarrow k+1}(x_{k+1}) = \alpha x_{k+1}$ for some $\alpha \in (0,1)$.
- Applying this T times gives: $\tilde{x}_0 = \alpha^T x_T$.
- Then $\mathrm{Var}\,(\tilde{x}_0) = \alpha^{2T}\,\mathrm{Var}\,(x_T)$.
- For small β, α ≈ 1−β/2, so $\alpha^{2T} \approx e^{-\beta T}$. If T is large compared to 1/β, this becomes tiny.

Hints for (d)

- For the stochastic reverse:

$$\hat{x}_k = \alpha \hat{x}_{k+1} + \sqrt{\gamma}\,\eta_k$$

- with α from (b).
- The variance recursion:

$$v_k := \mathrm{Var}\,(\hat{x}_k) = \alpha^2 v_{k+1} + \gamma.$$

- Solve by unrolling:

$$v_0 = \alpha^{2T} v_T + \gamma \sum_{j=0}^{T-1} \alpha^{2j}.$$

- Compare this to the **forward** variance formula; the sums will look similar.
- Choose γ so that the two expressions "mirror" each other, giving $v_0 \approx \sigma_0^2$.
- Intuitively, γ should be of the same order as β, because forward noise is β each step in this parameterization.

✅ **Fully Worked Solution**

We'll use shorthand:

- $a = \sqrt{1-\beta}$

□ ,

□ $b = \sqrt{\beta}$

- .

So the forward process is:

$$x_{k+1} = a x_k + b \epsilon_k.$$

(a) Forward mean and variance

Mean:

Given $x_0$has mean 0, and each ε_k has mean 0:

$$\mathbb{E}[x_{k+1}] = a\mathbb{E}[x_k] + b\mathbb{E}[\epsilon_k] = a\mathbb{E}[x_k].$$

With $\mathbb{E}[x_0] = 0$, we get by induction:

$$\mathbb{E}[x_k] = a^k \cdot 0 = 0.$$

So **all means are 0**.

Variance:

Denote $v_k := \mathrm{Var}\,(x_k)$. Using independence of x_k and ϵ_k:

$$v_{k+1} = \mathrm{Var}\,(ax_k + b\epsilon_k) = a^2 v_k + b^2.$$

We have:

- $a^2 = 1 - \beta$,
- $b^2 = \beta$.

So:

$$v_{k+1} = (1-\beta)v_k + \beta, v_0 = \sigma_0^2.$$

This is a linear recurrence. Let's solve it.

Standard solution of:

$$v_{k+1} - (1-\beta)v_k = \beta$$

has:

$$v_k = (1-\beta)^k \sigma_0^2 + \beta \sum_{j=0}^{k-1} (1-\beta)^j.$$

We recognize the sum as a finite geometric series:

$$\sum_{j=0}^{k-1} (1-\beta)^j = \frac{1-(1-\beta)^k}{1-(1-\beta)} = \frac{1-(1-\beta)^k}{\beta}.$$

Thus:

$$v_k = (1-\beta)^k \sigma_0^2 + \beta \cdot \frac{1-(1-\beta)^k}{\beta} = (1-\beta)^k \sigma_0^2 + [1-(1-\beta)^k].$$

So:

$$\boxed{\text{Var}\,(x_k) = (1-\beta)^k \sigma_0^2 + (1-(1-\beta)^k)}$$

For k = T:

$$\boxed{\text{Var}\,(x_T) = (1-\beta)^T \sigma_0^2 + (1-(1-\beta)^T)}$$

Approximation for small σ_0^2 and large T:

- If T is large and β>0, then $(1-\beta)^T \to 0$.
- So the first term vanishes, and the second tends to 1.

If $\sigma_0^2 \ll 1$ and T is large enough that $(1-\beta)^T \approx 0$, then:

$$\text{Var}\,(x_T) \approx 1.$$

This matches the intuition: the forward process **forgets** the original variance and tends toward a stationary variance of 1.

(b) Reverse conditional at stationarity

We now assume the process has reached its **stationary distribution**, so:

$$\text{Var}\,(x_k) = \text{Var}\,(x_{k+1}) = v_*.$$

From the recurrence:

$$v_* = a^2 v_* + b^2 \Rightarrow v_*(1-a^2) = b^2.$$

But:

- $a^2 = 1 - \beta$,
- $1 - a^2 = \beta$,
- $b^2 = \beta$.

So:

$$v_* \cdot \beta = \beta \Rightarrow v_* = 1.$$

So at stationarity, $x_k \sim \mathcal{N}(0,1)$for all k.

Now consider the joint Gaussian vector (x_k, x_{k+1}). We know:

$$x_{k+1} = ax_k + b\epsilon_k,$$

with:

- $\mathbb{E}[x_k] = 0$,
- $\mathbb{E}[x_{k+1}] = 0$,
- $\mathrm{Var}\,(x_k) = \mathrm{Var}\,(x_{k+1}) = 1$.

Compute covariance:

$$\mathrm{Cov}\,(x_k, x_{k+1}) = \mathbb{E}[x_k x_{k+1}] = \mathbb{E}[x_k(ax_k + b\epsilon_k)] = a\mathbb{E}[x_k^2] + b\mathbb{E}[x_k\epsilon_k] = a \cdot 1 + b \cdot 0 = a.$$

So the joint covariance matrix is:

$$\Sigma = \begin{pmatrix} 1 & a \\ a & 1 \end{pmatrix}.$$

Gaussian conditioning formula:

For jointly Gaussian $(X, Y)$with:

- Var(X) = σ_X^2,
- Var(Y) = σ_Y^2,
- Cov(X,Y) = σ_{XY},

we have:

$$X \mid Y = y \sim \mathcal{N}\left(\frac{\sigma_{XY}}{\sigma_Y^2}\, y,\, \sigma_X^2 - \frac{\sigma_{XY}^2}{\sigma_Y^2}\right).$$

Apply with:

- $X = x_k$,
- $Y = x_{k+1}$,
- $\sigma_X^2 = \sigma_Y^2 = 1$,
- $\sigma_{XY} = a$.

Thus:

$$x_k \mid x_{k+1} \sim \mathcal{N}(a x_{k+1},\, 1 - a^2).$$

But:

- $a = \sqrt{1-\beta}$
- ,
- $1 - a^2 = 1 - (1-\beta) = \beta$.

So we obtain:

$$\boxed{\mu_{k \leftarrow k+1}(x_{k+1}) = a\, x_{k+1} = \sqrt{1-\beta}\, x_{k+1}}$$
$$\boxed{\sigma_{k \leftarrow k+1}^2 = 1 - a^2 = \beta}$$

This is a nice symmetric result: **the reverse conditional has the same noise variance β**.

(c) Deterministic reverse: variance collapse

Now, define the deterministic reverse:

$$\tilde{x}_k = \mu_{k \leftarrow k+1}(\tilde{x}_{k+1}) = a\, \tilde{x}_{k+1}.$$

Apply repeatedly:

- $\tilde{x}_{T-1} = a\tilde{x}_T$,
- $\tilde{x}_{T-2} = a\tilde{x}_{T-1} = a^2\tilde{x}_T$,
- ...
- $\tilde{x}_0 = a^T\tilde{x}_T$.

With initial $\tilde{x}_T = x_T$:

$$\boxed{\tilde{x}_0 = a^T x_T.}$$

Assuming at stationarity we have $x_T \sim \mathcal{N}(0,1)$, then:

$$\mathrm{Var}\,(\tilde{x}_0) = a^{2T}\mathrm{Var}\,(x_T) = a^{2T} \cdot 1 = (1-\beta)^T.$$

For **small β and large T**, use:

$$(1-\beta)^T \approx e^{-\beta T}.$$

If T is on the order of $1/\beta$ or larger, this becomes:

- $e^{-\beta T} \ll 1$,
- so $\mathrm{Var}\,(\tilde{x}_0)$ is **very small**.

Meanwhile, the original data variance σ_0^2 was some O(1) number (possibly $\ll 1$ but still much larger than e^{-βT}). So the deterministic reverse maps the noisy sample back to something with **vanishing variance**, i.e. samples collapse near zero.

This is exactly **variance collapse**: without injecting noise in the reverse process, you over-denoise and kill almost all diversity.

(d) Stochastic reverse with constant γ

We now define:

$$\hat{x}_k = a\hat{x}_{k+1} + \sqrt{\gamma}\,\eta_k, \eta_k \sim \mathcal{N}(0,1),$$

with the same contraction factor a = $\sqrt{(1-\beta)}$. Start from:

$$\hat{x}_T \sim \mathcal{N}(0, \sigma_T^2)$$

with σ_T^2 from (a).

(1) Recursion for Var($\hat{x}_k$)

Let $v_k = \mathrm{Var}\,(\hat{x}_k)$. Then:

$$v_k = \mathrm{Var}\,(a\hat{x}_{k+1} + \sqrt{\gamma}\,\eta_k) = a^2 v_{k+1} + \gamma,$$

since $\hat{x}_{k+1}$and $\eta_k$are independent and Var(η_k)=1.

So:

$$\boxed{v_k = a^2 v_{k+1} + \gamma, v_T = \sigma_T^2.}$$

---

(2) Solving the recursion

We unroll from k = T−1 down to 0:

$$\begin{aligned}
v_{T-1} &= a^2 v_T + \gamma, \\
v_{T-2} &= a^2 v_{T-1} + \gamma = a^4 v_T + a^2\gamma + \gamma, \\
v_{T-3} &= a^2 v_{T-2} + \gamma = a^6 v_T + a^4\gamma + a^2\gamma + \gamma, \\
&\vdots \\
v_0 &= a^{2T} v_T + \gamma \sum_{j=0}^{T-1} a^{2j}\,.
\end{aligned}$$

Thus:

$$\boxed{v_0 = a^{2T}\sigma_T^2 + \gamma \sum_{j=0}^{T-1} a^{2j}\,.}$$

Now recall the forward variance from (a):

$$\sigma_T^2 = (1-\beta)^T \sigma_0^2 + (1 - (1-\beta)^T) = a^{2T}\sigma_0^2 + (1 - a^{2T}).$$

---

(3) Choose γ to recover σ_0^2 approximately

Plug σ_T^2 into v_0:

$$\begin{aligned} v_0 &= a^{2T}[a^{2T}\sigma_0^2 + (1 - a^{2T})] + \gamma \sum_{j=0}^{T-1} a^{2j} \\ &= a^{4T}\sigma_0^2 + a^{2T}(1 - a^{2T}) + \gamma \sum_{j=0}^{T-1} a^{2j}. \end{aligned}$$

We would like $v_0 \approx \sigma_0^2$.

Now, in typical diffusion usage:

- T is large enough that $a^{2T} = (1 - \beta)^T$ is very small.
- So we can approximate:
  - $a^{4T} \approx 0$,
  - $a^{2T}(1 - a^{2T}) \approx 0$.

Under that approximation:

$$v_0 \approx \gamma \sum_{j=0}^{T-1} a^{2j}.$$

But:

$$\sum_{j=0}^{T-1} a^{2j} = \frac{1 - a^{2T}}{1 - a^2} \approx \frac{1}{1 - a^2} \text{ if } a^{2T} \approx 0.$$

Recall:

- $a^2 = 1 - \beta$,
- so $1 - a^2 = \beta$.

Therefore:

$$\sum_{j=0}^{T-1} a^{2j} \approx \frac{1}{\beta}.$$

So:

$$v_0 \approx \gamma \cdot \frac{1}{\beta}.$$

To get $v_0 \approx \sigma_0^2$, we set approximately:

$$\boxed{\gamma \approx \beta\, \sigma_0^2.}$$

Interpretation:

- Forward process adds noise with variance β each step (in the x scale), which drives the variance to 1.
- Reverse process needs to add noise with variance ≈ β σ_0^2 each step (in this simple symmetric version) to unwind that effect and land back at Var ≈ σ_0^2.

In **real DDPMs**, the noise schedule is time-varying and more carefully derived. But this toy derivation shows the key idea:

The reverse noise variance per step must be **of the same order as the forward noise** (β), and when integrated over T steps, it should reconstruct the original data variance rather than collapsing to zero.

If you want, next we can do:

- another Level 3 on **KL vs DDPM objective**, or
- a Level 2 on **training the noise-predictor network** for diffusion.